第四部分

高级主题

第 13 章

显 式 锁

在 Java 5.0 之前，在协调对共享对象的访问时可以使用的机制只有 synchronized 和 volatile。Java 5.0 增加了一种新的机制：ReentrantLock。与之前提到过的机制相反，ReentrantLock 并不是一种替代内置加锁的方法，而是当内置加锁机制不适用时，作为一种可选择的高级功能。

13.1 Lock 与 ReentrantLock

在程序清单 13-1 给出的 Lock 接口中定义了一组抽象的加锁操作。与内置加锁机制不同的是，Lock 提供了一种无条件的、可轮询的、定时的以及可中断的锁获取操作，所有加锁和解锁的方法都是显式的。在 Lock 的实现中必须提供与内部锁相同的内存可见性语义，但在加锁语义、调度算法、顺序保证以及性能特性等方面可以有所不同。（第 14 章将介绍 Lock.newCondition。）

程序清单 13-1 Lock 接口

```
public interface Lock {
    void lock();
    void lockInterruptibly() throws InterruptedException;
    boolean tryLock();
    boolean tryLock(long timeout, TimeUnit unit)
        throws InterruptedException;
    void unlock();
    Condition newCondition();
}
```

ReentrantLock 实现了 Lock 接口，并提供了与 synchronized 相同的互斥性和内存可见性。在获取 ReentrantLock 时，有着与进入同步代码块相同的内存语义，在释放 ReentrantLock 时，同样有着与退出同步代码块相同的内存语义。（3.1 节以及第 16 章介绍内存可见性。）此外，与 synchronized 一样，ReentrantLock 还提供了可重入的加锁语义（请参见 2.3.2 节）。ReentrantLock 支持在 Lock 接口中定义的所有获取锁模式，并且与 synchronized 相比，它还为处理锁的不可用性问题提供了更高的灵活性。

为什么要创建一种与内置锁如此相似的新加锁机制？在大多数情况下，内置锁都能很好地工作，但在功能上存在一些局限性，例如，无法中断一个正在等待获取锁的线程，或者无法在请求获取一个锁时无限地等待下去。内置锁必须在获取该锁的代码块中释放，这就简化了编码工作，并且与异常处理操作实现了很好的交互，但却无法实现非阻塞结构的加锁规则。这些都是使用 synchronized 的原因，但在某些情况下，一种更灵活的加锁机制通常能提供更好的活跃性或性能。

程序清单 13-2 给出了 Lock 接口的标准使用形式。这种形式比使用内置锁复杂一些：必须在 finally 块中释放锁。否则，如果在被保护的代码中抛出了异常，那么这个锁永远都无法释放。当使用加锁时，还必须考虑在 try 块中抛出异常的情况，如果可能使对象处于某种不一致的状态，那么就需要更多的 try-catch 或 try-finally 代码块。（当使用某种形式的加锁时，包括内置锁，都应该考虑在出现异常时的情况。）

程序清单 13-2 使用 ReentrantLock 来保护对象状态

```
Lock lock = new ReentrantLock();
...
lock.lock();
try {
    // 更新对象状态
    // 捕获异常，并在必要时恢复不变性条件
} finally {
    lock.unlock();
}
```

如果没有使用 finally 来释放 Lock，那么相当于启动了一个定时炸弹。当“炸弹爆炸”时，将很难追踪到最初发生错误的位置，因为没有记录应该释放锁的位置和时间。这就是 ReentrantLock 不能完全替代 synchronized 的原因：它更加“危险”，因为当程序的执行控制离开被保护的代码块时，不会自动清除锁。虽然在 finally 块中释放锁并不困难，但也可能忘记。[⊖]

13.1.1 轮询锁与定时锁

可定时的与可轮询的锁获取模式是由 tryLock 方法实现的，与无条件的锁获取模式相比，它具有更完善的错误恢复机制。在内置锁中，死锁是一个严重的问题，恢复程序的唯一方法是

⊖ FindBugs 中有一个“未释放锁”的检查器，当一个锁没有在获取它的代码块之外的所有代码路径中释放，那么就找到它。

重新启动程序，而防止死锁的唯一方法就是在构造程序时避免出现不一致的锁顺序。可定时的与可轮询的锁提供了另一种选择：避免死锁的发生。

如果不能获得所有需要的锁，那么可以使用可定时的或可轮询的锁获取方式，从而使你重新获得控制权，它会释放已经获得的锁，然后重新尝试获取所有锁（或者至少会将这个失败记录到日志，并采取其他措施）。程序清单 13-3 给出了另一种方法来解决 10.1.2 节中动态顺序死锁的问题：使用 tryLock 来获取两个锁，如果不能同时获得，那么就回退并重新尝试。在休眠时间中包括固定部分和随机部分，从而降低发生活锁的可能性。如果在指定时间内不能获得所有需要的锁，那么 transferMoney 将返回一个失败状态，从而使该操作平缓地失败。（请参见 [CPJ 2.5.1.2] 和 [CPJ 2.5.1.3] 了解更多使用可轮询的锁来避免死锁的示例。）

程序清单 13-3 通过 tryLock 来避免锁顺序死锁

```
public boolean transferMoney(Account fromAcct,
                             Account toAcct,
                             DollarAmount amount,
                             long timeout,
                             TimeUnit unit)
        throws InsufficientFundsException, InterruptedException {
    long fixedDelay = getFixedDelayComponentNanos(timeout, unit);
    long randMod = getRandomDelayModulusNanos(timeout, unit);
    long stopTime = System.nanoTime() + unit.toNanos(timeout);

    while (true) {
        if (fromAcct.lock.tryLock()) {
            try {
                if (toAcct.lock.tryLock()) {
                    try {
                        if (fromAcct.getBalance().compareTo(amount)
                                < 0)
                            throw new InsufficientFundsException();
                        else {
                            fromAcct.debit(amount);
                            toAcct.credit(amount);
                            return true;
                        }
                    } finally {
                        toAcct.lock.unlock();
                    }
                }
            } finally {
                fromAcct.lock.unlock();
            }
        }
        if (System.nanoTime() < stopTime)
            return false;
        NANOSECONDS.sleep(fixedDelay + rnd.nextLong() % randMod);
    }
}
```

在实现具有时间限制的操作时，定时锁同样非常有用（请参见 6.3.7 节）。当在带有时间限制的操作中调用了一个阻塞方法时，它能根据剩余时间来提供一个时限。如果操作不能在指定

的时间内给出结果，那么就会使程序提前结束。当使用内置锁时，在开始请求锁后，这个操作将无法取消，因此内置锁很难实现带有时间限制的操作。

在程序清单 6-17 的旅游门户网站示例中，为询价的每个汽车租赁公司都创建了一个独立的任务。询价操作包含某种基于网络的请求机制，例如 Web 服务请求。但在询价操作中同样可能需要实现对紧缺资源的独占访问，例如通向公司的直连通信线路。

9.5 节介绍了确保对资源进行串行访问的方法：一个单线程的 Executor。另一种方法是使用一个独占锁来保护对资源的访问。程序清单 13-4 试图在 Lock 保护的共享通信线路上发送一条消息，如果不能在指定时间内完成，代码就会失败。定时的 tryLock 能够在这种带有时间限制的操作中实现独占加锁行为。

程序清单 13-4 带有时间限制的加锁

```
public boolean trySendOnSharedLine(String message,
                                   long timeout, TimeUnit unit)
                                   throws InterruptedException {
    long nanosToLock = unit.toNanos(timeout)
                     - estimatedNanosToSend(message);
    if (!lock.tryLock(nanosToLock, NANOSECONDS))
        return false;
    try {
        return sendOnSharedLine(message);
    } finally {
        lock.unlock();
    }
}
```

13.1.2 可中断的锁获取操作

正如定时的锁获取操作能在带有时间限制的操作中使用独占锁，可中断的锁获取操作同样能在可取消的操作中使用加锁。7.1.6 节给出了几种不能响应中断的机制，例如请求内置锁。这些不可中断的阻塞机制将使得实现可取消的任务变得复杂。lockInterruptibly 方法能够在获得锁的同时保持对中断的响应，并且由于它包含在 Lock 中，因此无须创建其他类型的不可中断阻塞机制。

可中断的锁获取操作的标准结构比普通的锁获取操作略微复杂一些，因为需要两个 try 块。（如果在可中断的锁获取操作中抛出了 InterruptedException，那么可以使用标准的 try-finally 加锁模式。）在程序清单 13-5 中使用了 lockInterruptibly 来实现程序清单 13-4 中的 sendOnSharedLine，以便在一个可取消的任务中调用它。定时的 tryLock 同样能响应中断，因此当需要实现一个定时的和可中断的锁获取操作时，可以使用 tryLock 方法。

程序清单 13-5 可中断的锁获取操作

```
public boolean sendOnSharedLine(String message)
        throws InterruptedException {
    lock.lockInterruptibly();
    try {
```

```
            return cancellableSendOnSharedLine(message);
        } finally {
            lock.unlock();
        }
    }

    private boolean cancellableSendOnSharedLine(String message)
        throws InterruptedException { ... }
```

13.1.3 非块结构的加锁

在内置锁中，锁的获取和释放等操作都是基于代码块的——释放锁的操作总是与获取锁的操作处于同一个代码块，而不考虑控制权如何退出该代码块。自动的锁释放操作简化了对程序的分析，避免了可能的编码错误，但有时侯需要更灵活的加锁规则。

在第 11 章中，我们看到了通过降低锁的粒度可以提高代码的可伸缩性。锁分段技术在基于散列的容器中实现了不同的散列链，以便使用不同的锁。我们可以通过采用类似的原则来降低链表中锁的粒度，即为每个链表节点使用一个独立的锁，使不同的线程能独立地对链表的不同部分进行操作。每个节点的锁将保护链接指针以及在该节点中存储的数据，因此当遍历或修改链表时，我们必须持有该节点上的这个锁，直到获得了下一个节点的锁，只有这样，才能释放前一个节点上的锁。在 [CPJ 2.5.1.4] 中介绍了使用这项技术的一个示例，并称之为连锁式加锁（Hand-Over-Hand Locking）或者锁耦合（Lock Coupling）。

13.2 性能考虑因素

当把 ReentrantLock 添加到 Java 5.0 时，它能比内置锁提供更好的竞争性能。对于同步原语来说，竞争性能是可伸缩性的关键要素：如果有越多的资源被耗费在锁的管理和调度上，那么应用程序得到的资源就越少。锁的实现方式越好，将需要越少的系统调用和上下文切换，并且在共享内存总线上的内存同步通信量也越少，而一些耗时的操作将占用应用程序的计算资源。

Java 6 使用了改进后的算法来管理内置锁，与在 ReentrantLock 中使用的算法类似，该算法有效地提高了可伸缩性。图 13-1 给出了在 Java 5.0 和 Java 6 版本中，内置锁与 ReentrantLock 之间的性能差异，测试程序的运行环境是 4 路的 Opteron 系统，操作系统为 Solaris。图中的曲线表示在某个 JVM 版本中 ReentrantLock 相对于内置锁的“加速比”。在 Java 5.0 中，ReentrantLock 能提供更高的吞吐量，但在 Java 6 中，二者的吞吐量非常接近[⊖]。这里使用了与 11.5 节相同的测试程序，而这次比较的是通过一个 HashMap 在由内置锁保护以及由 ReentrantLock 保护的情况下的吞吐量。

⊖ 这张曲线图中没有给出的信息是：Java 5.0 和 Java 6 之间的可伸缩性差异是源于内置锁的改进，而不是 ReentrantLock。

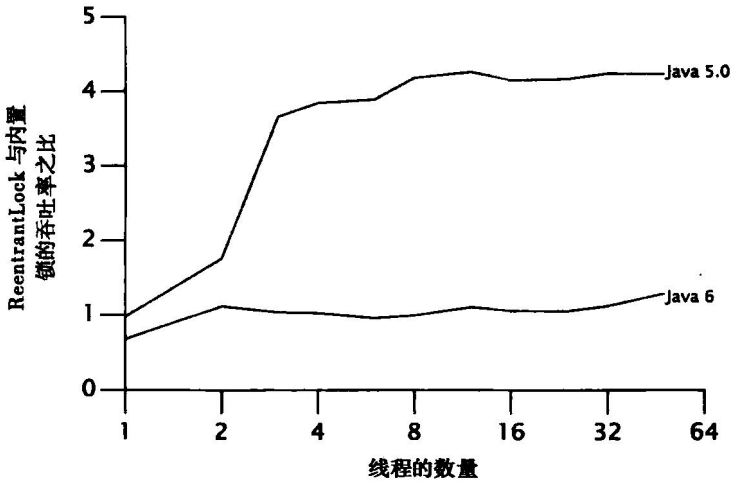

图 13-1 内置锁与 ReentrantLock 在 Java 5.0 与 Java 6 上的性能

在 Java 5.0 中，当从单线程（无竞争）变化到多线程时，内置锁的性能将急剧下降，而 ReentrantLock 的性能下降则更为平缓，因而它具有更好的可伸缩性。但在 Java 6 中，情况就完全不同了，内置锁的性能不会由于竞争而急剧下降，并且两者的可伸缩性也基本相当。

图 13-1 的曲线图告诉我们，像“X 比 Y 更快”这样的表述大多是短暂的。性能和可伸缩性对于具体平台等因素都较为敏感，例如 CPU、处理器数量、缓存大小以及 JVM 特性等，所有这些因素都可能会随着时间而发生变化。[⊖]

性能是一个不断变化的指标，如果在昨天的测试基准中发现 X 比 Y 更快，那么在今天就可能已经过时了。

13.3 公平性

在 ReentrantLock 的构造函数中提供了两种公平性选择：创建一个非公平的锁（默认）或者一个公平的锁。在公平的锁上，线程将按照它们发出请求的顺序来获得锁，但在非公平的锁上，则允许“插队”：当一个线程请求非公平的锁时，如果在发出请求的同时该锁的状态变为可用，那么这个线程将跳过队列中所有的等待线程并获得这个锁。（在 Semaphore 中同样可以选择采用公平的或非公平的获取顺序。）非公平的 ReentrantLock 并不提倡“插队”行为，但无法防止某个线程在合适的时候进行“插队”。在公平的锁中，如果有另一个线程持有这个锁或者有其他线程在队列中等待这个锁，那么新发出请求的线程将被放入队列中。在非公平的锁中，

⊖ 当开始写作本书时，ReentrantLock 似乎是解决锁的可伸缩性的最终手段。但不到一年的时间，内置锁在可伸缩性上已经获得了极大的提升。性能不仅是一个在不断变化的指标，而且变化得非常快。

只有当锁被某个线程持有时，新发出请求的线程才会被放入队列中[⊖]。

我们为什么不希望所有的锁都是公平的？毕竟，公平是一种好的行为，而不公平则是一种不好的行为，对不对？当执行加锁操作时，公平性将由于在挂起线程和恢复线程时存在的开销而极大地降低性能。在实际情况中，统计上的公平性保证——确保被阻塞的线程能最终获得锁，通常已经够用了，并且实际开销也小得多。有些算法依赖于公平的排队算法以确保它们的正确性，但这些算法并不常见。在大多数情况下，非公平锁的性能要高于公平锁的性能。

图 13-2 给出了 Map 的性能测试，并比较由公平的以及非公平的 ReentrantLock 包装的 HashMap 的性能，测试程序在一个 4 路的 Opteron 系统上运行，操作系统为 Solaris，在绘制结果曲线时采用了对数缩放比例[⊖]。从图中可以看出，公平性把性能降低了约两个数量级。不必要的话，不要为公平性付出代价。

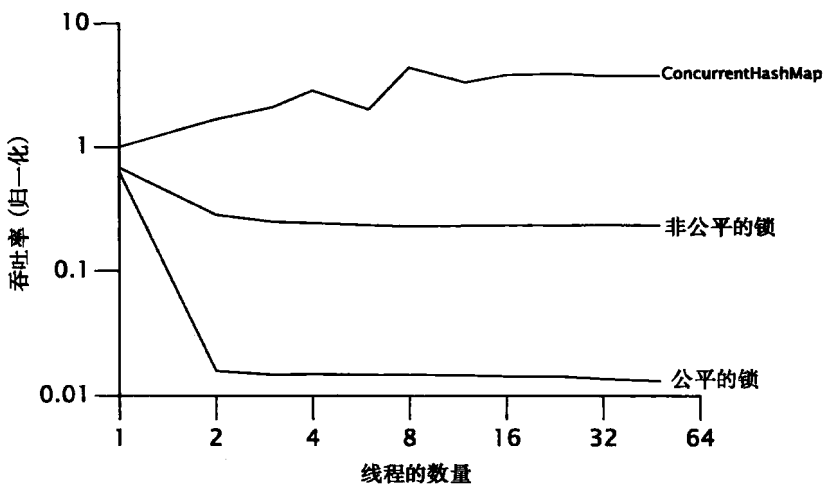

图 13-2 公平锁与非公平锁的性能比较

在激烈竞争的情况下，非公平锁的性能高于公平锁的性能的一个原因是：在恢复一个被挂起的线程与该线程真正开始运行之间存在着严重的延迟。假设线程 A 持有一个锁，并且线程 B 请求这个锁。由于这个锁已被线程 A 持有，因此 B 将被挂起。当 A 释放锁时，B 将被唤醒，因此会再次尝试获取锁。与此同时，如果 C 也请求这个锁，那么 C 很可能会在 B 被完全唤醒之前

⊖ 即使对于公平锁而言，可轮询的 tryLock 仍然会“插队”。

⊖ ConcurrentHashMap 的曲线在 4 个线程和 8 个线程之间的变动非常大。这些变动大多是测量噪声，噪声的可能来源包括：与元素散列码之间的偶然交互，线程的调度，重新调整映射集合的大小，垃圾回收或其他内存系统的作用，以及操作系统在测试用例运行时执行一些周期性的辅助任务。实际上，在性能测试中存在着各种各样的变动因素，而这些因素通常不需要进行控制。我们不要人为地使曲线变得平滑，因为在现实世界的性能测试中同样会存在各种各样的噪声。

获得、使用以及释放这个锁。这样的情况是一种“双赢”的局面：B 获得锁的时刻并没有推迟，C 更早地获得了锁，并且吞吐量也获得了提高。

当持有锁的时间相对较长，或者请求锁的平均时间间隔较长，那么应该使用公平锁。在这些情况下，“插队”带来的吞吐量提升（当锁处于可用状态时，线程却还处于被唤醒的过程中）则可能不会出现。

与默认的 ReentrantLock 一样，内置加锁并不会提供确定的公平性保证，但在大多数情况下，在锁实现上实现统计上的公平性保证已经足够了。Java 语言规范并没有要求 JVM 以公平的方式来实现内置锁，而在各种 JVM 中也没有这样做。ReentrantLock 并没有进一步降低锁的公平性，而只是使一些已经存在的内容更明显。

13.4 在 synchronized 和 ReentrantLock 之间进行选择

ReentrantLock 在加锁和内存上提供的语义与与内置锁相同，此外它还提供了一些其他功能，包括定时的锁等待、可中断的锁等待、公平性，以及实现非块结构的加锁。ReentrantLock 在性能上似乎优于内置锁，其中在 Java 6 中略有胜出，而在 Java 5.0 中则是远远胜出。那么为什么不放弃 synchronized，并在所有新的并发代码中都使用 ReentrantLock？事实上有些作者已经建议这么做，将 synchronized 作为一种“遗留”结构，但这会将好事情变坏。

与显式锁相比，内置锁仍然具有很大的优势。内置锁为许多开发人员所熟悉，并且简洁紧凑，而且在许多现有的程序中都已经使用了内置锁——如果将这两种机制混合使用，那么不仅容易令人困惑，也容易发生错误。ReentrantLock 的危险性比同步机制要高，如果忘记在 finally 块中调用 unlock，那么虽然代码表面上能正常运行，但实际上已经埋下了一颗定时炸弹，并很有可能伤及其他代码。仅当内置锁不能满足需求时，才可以考虑使用 ReentrantLock。

> ***在一些内置锁无法满足需求的情况下，ReentrantLock 可以作为一种高级工具。当需要一些高级功能时才应该使用 ReentrantLock，这些功能包括：可定时的、可轮询的与可中断的锁获取操作，公平队列，以及非块结构的锁。否则，还是应该优先使用 synchronized。***

在 Java 5.0 中，内置锁与 ReentrantLock 相比还有另一个优点：在线程转储中能给出在哪些调用帧中获得了哪些锁，并能够检测和识别发生死锁的线程。JVM 并不知道哪些线程持有 ReentrantLock，因此在调试使用 ReentrantLock 的线程的问题时，将起不到帮助作用。Java 6 解决了这个问题，它提供了一个管理和调试接口，锁可以通过该接口进行注册，从而与 ReentrantLocks 相关的加锁信息就能出现在线程转储中，并通过其他的管理接口和调试接口来访问。与 synchronized 相比，这些调试消息是一种重要的优势，即便它们大部分都是临时性消息，线程转储中的加锁能给很多程序员带来帮助。ReentrantLock 的非块结构特性仍然意味着，获取锁的操作不能与特定的栈帧关联起来，而内置锁却可以。

未来更可能会提升 synchronized 而不是 ReentrantLock 的性能。因为 synchronized 是 JVM 的内置属性，它能执行一些优化，例如对线程封闭的锁对象的锁消除优化，通过增加锁的粒度

来消除内置锁的同步（请参见 11.3.2 节），而如果通过基于类库的锁来实现这些功能，则可能性不大。除非将来需要在 Java 5.0 上部署应用程序，并且在该平台上确实需要 ReentrantLock 包含的可伸缩性，否则就性能方面来说，应该选择 synchronized 而不是 ReentrantLock。

13.5 读－写锁

ReentrantLock 实现了一种标准的互斥锁：每次最多只有一个线程能持有 ReentrantLock。但对于维护数据的完整性来说，互斥通常是一种过于强硬的加锁规则，因此也就不必要地限制了并发性。互斥是一种保守的加锁策略，虽然可以避免“写 / 写”冲突和“写 / 读”冲突，但同样也避免了“读 / 读”冲突。在许多情况下，数据结构上的操作都是“读操作”——虽然它们也是可变的并且在某些情况下被修改，但其中大多数访问操作都是读操作。此时，如果能够放宽加锁需求，允许多个执行读操作的线程同时访问数据结构，那么将提升程序的性能。只要每个线程都能确保读取到最新的数据，并且在读取数据时不会有其他的线程修改数据，那么就不会发生问题。在这种情况下就可以使用读 / 写锁：一个资源可以被多个读操作访问，或者被一个写操作访问，但两者不能同时进行。

在程序清单 13-6 的 ReadWriteLock 中暴露了两个 Lock 对象，其中一个用于读操作，而另一个用于写操作。要读取由 ReadWriteLock 保护的数据，必须首先获得读取锁，当需要修改 ReadWriteLock 保护的数据时，必须首先获得写入锁。尽管这两个锁看上去是彼此独立的，但读取锁和写入锁只是读－写锁对象的不同视图。

程序清单 13-6 ReadWriteLock 接口

```
public interface ReadWriteLock {
    Lock readLock();
    Lock writeLock();
}
```

在读－写锁实现的加锁策略中，允许多个读操作同时进行，但每次只允许一个写操作。与 Lock 一样，ReadWriteLock 可以采用多种不同的实现方式，这些方式在性能、调度保证、获取优先性、公平性以及加锁语义等方面可能有所不同。

读－写锁是一种性能优化措施，在一些特定的情况下能实现更高的并发性。在实际情况中，对于在多处理器系统上被频繁读取的数据结构，读 - 写锁能够提高性能。而在其他情况下，读－写锁的性能比独占锁的性能要略差一些，这是因为它们的复杂性更高。如果要判断在某种情况下使用读－写锁是否会带来性能提升，最好对程序进行分析。由于 ReadWriteLock 使用 Lock 来实现锁的读－写部分，因此如果分析结果表明读－写锁没有提高性能，那么可以很容易地将读－写锁换为独占锁。

在读取锁和写入锁之间的交互可以采用多种实现方式。ReadWriteLock 中的一些可选实现包括：

释放优先。当一个写入操作释放写入锁时，并且队列中同时存在读线程和写线程，那么应该优先选择读线程，写线程，还是最先发出请求的线程？

读线程插队。如果锁是由读线程持有，但有写线程正在等待，那么新到达的读线程能否立即获得访问权，还是应该在写线程后面等待？如果允许读线程插队到写线程之前，那么将提高并发性，但却可能造成写线程发生饥饿问题。

重入性。读取锁和写入锁是否是可重入的？

降级。如果一个线程持有写入锁，那么它能否在不释放该锁的情况下获得读取锁？这可能会使得写入锁被“降级”为读取锁，同时不允许其他写线程修改被保护的资源。

升级。读取锁能否优先于其他正在等待的读线程和写线程而升级为一个写入锁？在大多数的读－写锁实现中并不支持升级，因为如果没有显式的升级操作，那么很容易造成死锁。（如果两个读线程试图同时升级为写入锁，那么二者都不会释放读取锁。）

ReentrantReadWriteLock 为这两种锁都提供了可重入的加锁语义。与 ReentrantLock 类似，ReentrantReadWriteLock 在构造时也可以选择是一个非公平的锁（默认）还是一个公平的锁。在公平的锁中，等待时间最长的线程将优先获得锁。如果这个锁由读线程持有，而另一个线程请求写入锁，那么其他读线程都不能获得读取锁，直到写线程使用完并且释放了写入锁。在非公平的锁中，线程获得访问许可的顺序是不确定的。写线程降级为读线程是可以的，但从读线程升级为写线程则是不可以的（这样做会导致死锁）。

与 ReentrantLock 类似的是，ReentrantReadWriteLock 中的写入锁只能有唯一的所有者，并且只能由获得该锁的线程来释放。在 Java 5.0 中，读取锁的行为更类似于一个 Semaphore 而不是锁，它只维护活跃的读线程的数量，而不考虑它们的标识。在 Java 6 中修改了这个行为：记录哪些线程已经获得了读者锁。[⊖]

当锁的持有时间较长并且大部分操作都不会修改被守护的资源时，那么读－写锁能提高并发性。在程序清单 13-7 的 ReadWriteMap 中使用了 ReentrantReadWriteLock 来包装 Map，从而使它能在多个读线程之间被安全地共享，并且仍然能避免“读－写”或“写－写”冲突[⊖]。在现实中，ConcurrentHashMap 的性能已经很好了，因此如果只需要一个并发的基于散列的映射，那么就可以使用 ConcurrentHashMap 来代替这种方法，但如果需要对另一种 Map 实现（例如 LinkedHashMap）提供并发性更高的访问，那么可以使用这项技术。

程序清单 13-7 用读－写锁来包装 Map

```
public class ReadWriteMap<K,V> {
    private final Map<K,V> map;
    private final ReadWriteLock lock = new ReentrantReadWriteLock();
    private final Lock r = lock.readLock();
    private final Lock w = lock.writeLock();

    public ReadWriteMap(Map<K,V> map) {
        this.map = map;
    }
```

⊖ 做出这种修改的一个原因是：在 Java 5.0 的锁实现中，无法区别一个线程是首次请求读取锁，还是可重入锁请求，从而可能使公平的读－写锁发生死锁。

⊖ ReadWriteMap 并没有实现 Map，因为实现一些方法（例如 entrySet 和 values）是非常困难的，况且“简单”的方法通常已经足够了。

```
    public V put(K key, V value) {
        w.lock();
        try {
            return map.put(key, value);
        } finally {
            w.unlock();
        }
    }
    // 对 remove(), putAll(), clear() 等方法执行相同的操作

    public V get(Object key) {
        r.lock();
        try {
            return map.get(key);
        } finally {
            r.unlock();
        }
    }
    // 对其他只读的 Map 方法执行相同的操作
}
```

图 13-3 给出了分别用 ReentrantLock 和 ReadWriteLock 来封装 ArrayList 的吞吐量比较，测试程序在 4 路的 Opteron 系统上运行，操作系统为 Solaris。这里使用的测试程序与本书使用的 Map 性能测试基本类似——每个操作随机地选择一个值并在容器中查找这个值，并且只有少量的操作会修改这个容器中的内容。

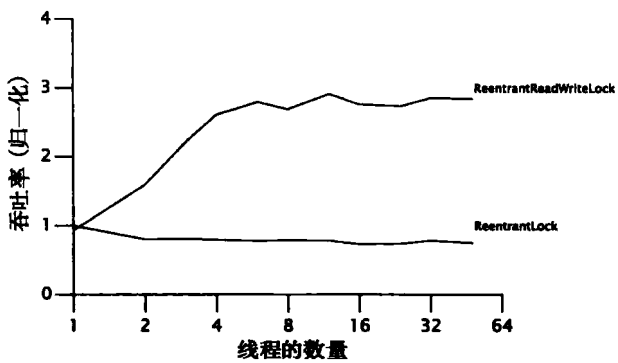

图 13-3 读－写锁的性能

小结

与内置锁相比，显式的 Lock 提供了一些扩展功能，在处理锁的不可用性方面有着更高的灵活性，并且对队列行有着更好的控制。但 ReentrantLock 不能完全替代 synchronized，只有在 synchronized 无法满足需求时，才应该使用它。

读－写锁允许多个读线程并发地访问被保护的对象，当访问以读取操作为主的数据结构时，它能提高程序的可伸缩性。